arXiv:2405.08344v1 [cs.CV] 14 May 2024

No Time to Waste: Squeeze Time into Channel for Mobile Video Understanding

Yingjie Zhai, Wenshuo Li, Yehui Tang, Xinghao Chen*, Yunhe Wang*
Huawei Noah's Ark Lab.
{xinghao.chen,yunhe.wang}@huawei.com

Abstract

Current architectures for video understanding mainly build upon 3D convolutional blocks or 2D convolutions with additional operations for temporal modeling. However, these methods all regard the temporal axis as a separate dimension of the video sequence, which requires large computation and memory budgets and thus limits their usage on mobile devices. In this paper, we propose to squeeze the time axis of a video sequence into the channel dimension and present a lightweight video recognition network, term as *SqueezeTime*, for mobile video understanding. To enhance the temporal modeling capability of the proposed network, we design a Channel-Time Learning (CTL) Block to capture temporal dynamics of the sequence. This module has two complementary branches, in which one branch is for temporal importance learning and another branch with temporal position restoring capability is to enhance inter-temporal object modeling ability. The proposed SqueezeTime is much lightweight and fast with high accuracies for mobile video understanding. Extensive experiments on various video recognition and action detection benchmarks, *i.e.*, Kinetics400, Kinetics600, HMDB51, AVA2.1 and THUMOS14, demonstrate the superiority of our model. For example, our SqueezeTime achieves $+1.2\%$ accuracy and $+80\%$ GPU throughput gain on Kinetics400 than prior methods. Codes are publicly available at https://github.com/xinghaochen/SqueezeTime and https://github.com/mindspore-lab/models/tree/master/research/huawei-noah/SqueezeTime.

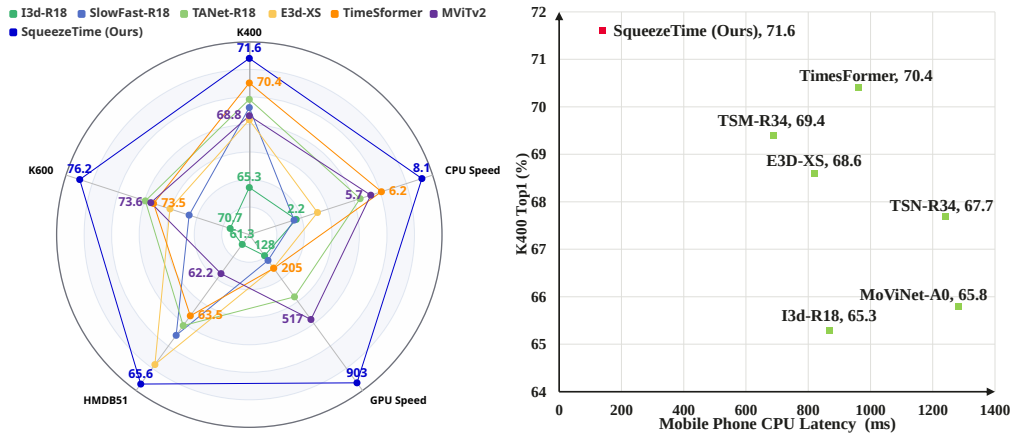
1 Introduction

In recent years, the amount of videos is of explosive growth. Different from image recognition, processing these video data is much more resource-consuming. For example, when simply extending the image recognition model, *e.g.*, ResNet [1], into the temporal-spatial version, the multiply-add operations are increased by 27 times [2], which is much compute-demanding. Therefore, research on how to design efficient video understanding models is becoming a hot topic, which has many applications, *e.g.*, mobile video analysis, autonomous driving, robotics, industrial control, *etc*.

However, it nowadays remains difficult to find models that can run fast on edge devices with high accuracy. Traditional 3D convolutional networks, *e.g.*, 3D ConvNets [6], I3D [7] and SlowFast [8] can jointly learn spatial and temporal features from videos but consume large amounts of memory and computation, which makes them not suitable for the mobile usage. To make the 3D convolutional network more efficient, some previous methods tried to improve the 3D convolutional network by 2D decomposition or approximation manually [2, 9–11]. These manually designed methods cost massive effort and time, thus a series of other methods [12, 13] used the Neural Architecture Search (NAS) technique to automatically design 3D video recognition architectures. However, searching for such 3D architectures on a video benchmark is also time-consuming (a lot of days on GPUs

*Corresponding authors

Preprint. Under review.



(a) (b)

Figure 1: (a) Performance comparison for mobile video recognition of multiple methods on K400 [3], K600 [4] and HMDB51 [5] datasets. We report Top1 accuracy (%), GPU Speed (throughput, videos / s), and CPU Speed (videos / s) on the figure. Note the CPU speed is measured by the '1 / latency (ms)' for better visualization. (b) Latency comparison of multiple models on a mobile device.

or TPUs) or handware-dependent [13]. Another kind of popular methods introduce the 2D CNN into the video recognition task by incorporating extra temporal learning mechanisms, *e.g.*, Temporal Shift Module [14], Temporal Difference Moudle [15], Temporal Adaptive Module [16], Adaptive Focus [17], Temporal Patch Shift [18], Temporally-Adaptive Convolutions [19], *etc*. Though these methods have improved running speed, the accuracies are not quite satisfactory in mobile settings. Recently, transformers-based models [20–24] are proposed for video analysis with encouraging performance on various datasets, but they are not friendly to mobile devices.

Note that the above models treat the temporal axis of videos as an extra dimension, and keep the original temporal dimension when forwarding the video feature (see Figure 2). Such kinds of operations cost large amounts of memory and need extra computation costs for the extra dimension, which is not efficient enough for mobile computing. In this paper, we reveal that it is not necessary to keep the temporal axis to build a backbone network for mobile video tasks. Therefore, we propose to squeeze the temporal axis of the video data into the spatial channel dimension and design a corresponding lightweight backbone, *i.e.*, ***SqueezeTime***, for mobile video understanding. It is demonstrated fast with high accuracies on both GPU and (Mobile) CPU devices for video recognition (please see Figure 1). To eliminate the negative impact caused by the squeezing operation, we propose a Channel-Time Learning Block (CTL) to learn the temporal dynamics embedded into the channels. Specifically, the CTL block contains two branches: one branch with **T**emporal **F**ocus **C**onvolution (TFC) concentrates on learning the potential temporal importance of different channels, and another branch is leveraged to restore the temporal information of multiple channels and to model the **I**nter-temporal **O**bject **I**nteraction (IOI) using large kernels. Built with these CTL blocks, the proposed *SqueezeTime* can capture strong video representations using only a 2D CNN with no extra resource consumption brought by the temporal dimension, which is much more efficient for mobile video analysis. Note most previous work concentrates on designing models with higher performance using large computation costs (> 40 GFLOPs), which is not suitable for comparing mobile video recognition. Thus, in this work, we also make experiments to evaluate the lightweight version of popular models for mobile video analysis, which demonstrates the proposed *SqueezeTime* can exceed other state-of-the-art methods on various datasets.

In summary, the key contributions of our work are:

- We propose to squeeze the temporal dimension of the video sequence into spatial channels, which is much faster with low memory-consuming and low computation cost.
- We elaborately design the *SqueezeTime* with CTL blocks. The CTL can learn the potential temporal importance of channels, restore temporal information, and enhance inter-temporal object modeling ability, which brings 4.4% Top1 accuracy gain on K400.
- We thoroughly benchmark popular video models in the mobile video analysis settings. Extensive experiments demonstrate the proposed *SqueezeTime* can yield higher accuracy ($+1.2\%$ Top1 on K400) with faster CPU and GPU ($+80\%$ throughput on K400) speed.

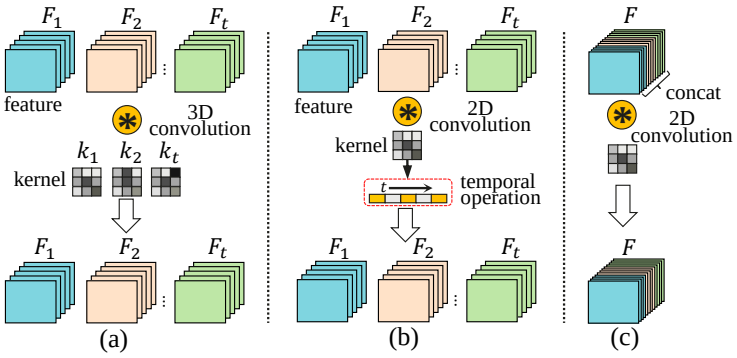


Figure 2: Feature and kernel illustration of different video models, *i.e.*, (a) 3D CNN-based models, (b) 2D CNN with temporal modeling operations, and (c) the proposed squeeze time mechanism.

2 Related Work

CNN-based Methods. For video recognition models, it is important to capture temporal relations between video frames [25]. CNN-based methods can be divided into two categories to address the problem [26]. One kind of method intuitively designs efficient video recognition networks by stacking 3D convolution [2, 6–8, 12, 13, 27], which is used to directly extract spatial-temporal representations from video clips [9, 27]. For example, X3D [2] proposed a family of 3D CNNs by expanding a tiny 2D image classifier along multiple network axes, *i.e.*, space, time, width, and depth. MoViNet [12] designed a three-step approach to improve computational efficiency while reducing the peak memory usage of 3D CNNs. E3D [13] derived an analytic entropy searching strategy to automatically design 3D CNN architectures. Another kind of method designs efficient video recognition models by equipping the 2D CNNs with the temporal modeling capacity [11, 14–16, 19, 28, 28–35]. For example, Lin *et al.* [14] proposed a generic and effective temporal shift module (TSM), which can be inserted into 2D CNNs at zero computation and zero parameters. Liu *et al.* [16] presented a novel temporal adaptive module (TAM) to produce video-specific temporal kernels based on feature maps to capture diverse motion patterns. Huang *et al.* [19] designed a temporal-adaptive convolution (TAdaConv) for video understanding.

Transformer-based Methods. Recently, vision transformers are introduced to video understanding [18, 20–23, 36–41]. For example, ViViT [36] presented pure-transformer models by factorizing the model along spatial and temporal dimensions to increase efficiency and scalability. Video Swin Transformer [22] designed an inductive bias of locality in video transformers to get a better speed-accuracy trade-off. MViTv2 [21] incorporated decomposed relative positional embeddings and residual pooling connections into the transformer for video recognition. TimeSformer [20] suggested divided attention, which separately applies temporal attention and spatial attention with each block, leading to good performance. Although transformer-based methods achieve strong performance, they are not friendly to mobile usage.

Mobile Video Understanding. Most recent work is focused on designing efficient video recognition models to achieve higher accuracy on various benchmarks. But there are few works [2, 12, 13] that are devoted to designing lightweight models deployed on mobile devices. The recent work X3D [2], MoViNet [12], and E3D [13] based on stepwise network expansion or neural architecture searching designed several lightweight models, but the trade-off between the speed and accuracy is not satisfactory enough. Therefore, in this paper, we aim to build a lightweight video backbone with large GPU throughput, low CPU latency, and high accuracy for mobile video understanding.

3 SqueezeTime Networks

The core concept of our work, to design an efficient mobile recognition backbone, is first squeezing the temporal information into channels, which can save a lot of computing resources and memory consumption from the temporal dimension. Then we need to carefully design the model so that it can excavate important temporal dynamics from these fused channels. The framework of the proposed model is shown in Figure 3. Let $X \in R^{3,t,h,w}$ denotes the input video sequence, we first squeeze the temporal axis into channels by the reshape operation and get the squeezed video input $X' \in R^{3\times t,h,w}$. Then we feed it into the stem layer with 5×5 convolution to get the base feature $\mathbf{F}_b$, which is then processed by four stages of blocks to learn temporal dynamics and object representations. Finally, an average pooling layer followed by a fully-connected layer is used to predict the result. At each stage,

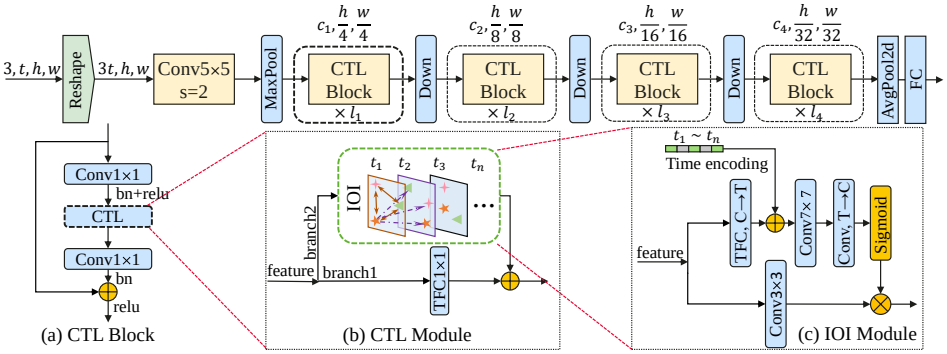


Figure 3: Pipeline of the proposed *SqueezeTime*. The input video clip is first reshaped by squeezing the temporal dimension into channels and is then fed into the following network. The proposed network contains four main stages, each stage with a stack of CTL Blocks, which are elaborately designed to excavate and restore hidden temporal representations.

we use a 2×2 convolution with a stride 2 to downsample larger-scale features. We will describe the proposed method in detail in the following section.

3.1 Squeeze and Restore Time

Previous CNN-based methods treat the temporal axis of the video sequence as an extra dimension, *e.g.*, 3D CNN-based methods [2, 7, 13] and 2D CNN with temporal learning modules [14, 16, 19]. Let $t, h, w, c_{in}, c_{out}, k$ represent the temporal size, spatial sizes, input channels, output channels, and kernel size of the input feature. The computation complexity of the 3D CNN-based methods and 2D CNN-based methods with temporal learning module are $2c_{out}c_{in}k^3hwt$ and $2c_{out}c_{in}k^2hwt + O(t)$, respectively. $O(t)$ represents the computation of temporal modeling. As shown in Figure 2, we propose to squeeze the temporal dimension into spatial channels, which can further reduce the computation complexity to $2c_{out}c_{in}k^2hw$, and more importantly, it can save a lot of memory originally occupied by keeping the temporal axis (please see Figure 2 to compare with other models). The proposed squeeze mechanism is formulated by:

$$\mathbf{F}_b = f_m(f_s(X)), \tag{1}$$

where f_s is the squeeze function, f_m is the mix up function, and $\mathbf{F}_b$ is the squeezed feature without temporal dimension. Note that the temporal information of the squeezed feature $\mathbf{F}_b$ is mixed up, which impedes the learning of discriminate features. For one thing, the temporal information is out of order. For another thing, the spatial objects from the same frame may be distributed in different temporal channels. Therefore, we propose to design a 2D block with temporal importance learning and inter-temporal object interaction function to remedy the problems. The restoration process is formulated as:

$$\mathbf{F}' = \beta(\mathbf{F}_b) + \xi(\mathbf{F}_b + \tau), \tag{2}$$

where β is the temporal importance learning function, ξ is the inter-temporal interaction function, and τ is the injected temporal order information. $\mathbf{F}'$ is the restored feature.

3.2 Channel-Time Learning (CTL) Block

Channel-Time Learning (CTL) Block is the basic component of *SqueezeTime* to implement the Formula 2. As shown in Figure 3 (a), the Channel-Time Learning (CTL) Block, built with CTL Module, follows the bottleneck form of ResNet [1]. It contains a 1×1 convolution to reduce the channels, a CTL module to learn temporal and spatial representations, and another 1×1 convolution to restore the channel number. The CTL block can be formulated by:

$$\mathbf{F}_o = \mathrm{Conv}_{1\times1}^{rC\to C}\left(\mathrm{CTL}\left(\mathrm{Conv}_{1\times1}^{C\to rC}(\mathbf{F}_i)\right)\right) + \mathbf{F}_i, \tag{3}$$

where $\mathbf{F}_i$ and $\mathbf{F}_o$ are the input feature and output feature of the CTL block, and r is the ratio controlling the channel expansion. $\mathrm{CTL}(\cdot)$ represents the CTL Module. Note we omit the batch normalization and ReLU operations in the formula. We set the reduction factor r to 0.25 as the default.

As shown in Figure 3 (a), the channel-time learning module contains two complementary branches. The bottom branch1 is a temporal focus convolution (TFC) with 1×1 kernel size to especially concentrate on capturing the temporal-channel importance. The top branch2 is an inter-temporal object interaction (IOI) module that aims to restore the temporal position information and model

the inter-channel spatial relations using large kernels. The final output of the CTL module is the summation of the two branches.

Temporal Focus Convolution (TFC). When squeezing the temporal dimension into the channels, a natural question is: whether the original 2D convolution is suitable to model the temporal representations hidden in different channels. For a common 2D convolution, it multiplies and accumulates the values of all channels in a local window size:

$$f(x,y) = \sum_{m=0}^{c} \sum_{i=0}^{k} \sum_{j=0}^{k} g(i,j,m) \times h(x-i, y-j, m), \tag{4}$$

where g and h represent the kernel and feature map respectively, $f(x,y)$ is the calculated output of the 2D convolution in spatial coordinate (x,y), k is the kernel size, and c is the channel number. Note the common 2D operation regards different channels as same importance when aggregating them. However, we argue that, when temporal information is squeezed into channels, it is necessary to distinguish their temporal importance. The improved **T**emporal **F**ocus 2D **C**onvolution (TFC) is formulated by:

$$f'(x,y) = \sum_{m=0}^{c} \sum_{i=0}^{k} \sum_{j=0}^{k} w_m \times g(i,j,m) \times h(x-i, y-j, m), \tag{5}$$

where w_m is the temporal-adaptive weights calculated according to the input features, it models the temporal importance of different channels. w_m can be computed using a lightweight module, *i.e.*, weight computation module. In this paper, we simply use a global MaxPool2d followed by a two-layer MLP as the WCM.

Inter-temporal Object Interaction Module. The IOI module is designed from two aspects of consideration. (1) On the one hand, when the temporal information of a video clip is squeezed into channels, and after being processed by a stem layer, the temporal order information of channels is mixed up and some discriminate details are lost. The model needs to restore such temporal details. (2) On the other hand, we argue that it is important to capture the relation among multiple objects attached to different temporal channels. Because, for the squeezed modality, two objects that interact with each other over time may appear in different channels. The model also needs to learn such spatial relations. IOI module is designed to restore such temporal positional dynamics and temporal object relations (right part of Formula 2):

$$\xi = \phi\left(\eta^{T\to C_o}\left(\varphi\left(\eta^{C_i\to T}\left(\mathbf{F}_b\right) + \mathbf{F_T}\right)\right)\right) \otimes \eta^{C_i\to C_o}\left(\mathbf{F}_b\right), \tag{6}$$

where ϕ is a sigmoid function, η is a mapping function that converts the number of channels, T is the number of frames, η is a function that excavates the inter-temporal object representations, $\mathbf{F_T}$ is the temporal position encoding, and $\otimes$ is the elementwise multiplication.

In this paper, we implement the above formula based on pure 2D convolution. As shown in Figure 3 (c), the IOI module consists of two sub-branches. The top branch first uses a 3×3 TFC to reduce the number of channels (C) to the number of frames (T) and to capture the temporal importance, simultaneously. Then temporal position encoding information is injected to restore the temporal dynamics. After this, a 7×7 convolution is leveraged to model the object relations between T frames. Note it can be replaced by other more efficient modules to capture the cross-temporal object interactions if possible in the future. The convolution with large kernels, in this work, is the simplest and also an effective way to model the cross-temporal object interactions. Finally, a 3×3 convolution is used to get the output number of channels. The bottom branch makes a direct mapping from input channels to output channels.

4 Experiments

4.1 Datasets

We evaluate the proposed method on 6 datasets in total. For action recognition, we conduct experiments on 4 commonly-used datasets including Kinetics400 (K400) [3], Kinetics600 (K600) [4], and HMDB51 [5]. K400 is a large-scale action recognition benchmark with 400 action classes, it contains $\sim 240k$ training videos and $\sim 20k$ validation videos. K600 is an extension of the K400 dataset, which includes 600 classes (each class with at least 600 clips). HMDB51 includes $6,849$ clips divided into 51 action categories, each with a minimum of 101 clips. For action detection, we experiment on AVA2.1 [43] dataset. AVA2.1 densely annotates 80 atomic actions by localizing them in space and time in 430 15-minute movie clips. It generates 1.62M action labels with per human multiple labels occurring frequently. Besides, we conduct experiments on THUMOS14 [44], which contains 200 and 212 untrimmed videos for training and testing.

Table 1: Ablation of two branches of CTL Module on K400 [3].

Settings	Param	FLOPs	Top1	Top5
base	23.6M	4.7G	67.2	87.1
TFConv Branch	16.1M	3.1G	62.8	83.5
IOI Branch	27.5M	5.3G	69.6	88.3
Ensemble (*SqueezeTime*)	28.7M	5.5G	**71.6**	**89.3**

Table 2: Ablation analysis of IOI branch of CTL Module on K400 [3] dataset.

#	IOI	TFC	TPosEmb	Param	FLOPs	Top1	Top5
1	×	×	×	23.6M	4.7G	67.2	87.1
2	✓	×	×	24.9M	5.3G	68.1	87.4
3	✓	✓	×	27.4M	5.3G	69.2	88.4
4	✓	✓	✓	27.5M	5.3G	69.6	88.3

Table 3: Ablation analysis of temporal focus convolution (TFC) of CTL Module on K400 [3].

Settings	Param	FLOPs	Top1	Top5
1×1 Convolution	13.5M	3.1G	49.9	73.3
Temporal focus conv	16.1M	3.1G	62.8	83.5

Table 4: Effect of the input frames of *SqueezeTime*.

Frame Number	4	8	16	32
FLOPs (G)	4.8	5.0	5.5	6.5
Top1 Acc (%)	70.7	70.9	71.6	71.7
Top5 Acc (%)	89.1	88.9	89.3	89.4

4.2 Implementation Details

In this work, we train and test the model using PyTorch framework on 8 NVIDIA Tesla V100 GPUs. Following the configurations of [1], we set the numbers of CTL blocks of the proposed *SqueezeTime* in four stages to [3, 4, 6, 3], and set the channel numbers of four stages to [256, 512, 1024, 2048]. The input frames of the proposed model are set to 16 for all experiments. For K400 [3] and K600 [4] datasets, we train the mode using SGD optimizer with CosineAnnealing learning rate. The initial learning rate is 0.015, the warm-up epoch number is set to 8, and weight decay is set to 7e-5. The model is initialized by pre-trained models on ImageNet1K [45]. The total epoch number is 100 and the batch size is 512. The resolution of the input frame is 224×224. Following the common practice [19], we use a dense sampling strategy with the frame interval of 4. Following [19, 21, 22], we use multi-scale cropping, random flip, color jitter, and random erasing to augment the training data. For testing, we use three spatial crops with 10 temporal clips (3×10) strategy to evaluate the model, and the shorter side of each video is resized to 224.

For the HMDB51 [5] dataset, we use a multi-step learning rate strategy with total epochs of 50, an initial learning rate of 0.015, and milestones of [12, 24]. The weight decay is set to 1e-2. We use 3 crops 2 clips strategy to evaluate the model. Others are the same as K400. For the AVA2.1 [43] dataset, we use SGD optimizer and set the total epoch number, initial learning rate, and weight decay to 50, 0.01, and 1e-5 respectively. Other settings are same as [8]. For the THUMOS14 [44], we use the default training settings as AFSD [46]. To compare with other methods in fair, we use the released codes of their paper or mmaction2 [47] framework to conduct experiments.

4.3 Ablation Study

Ablation analysis of two branches of CTL Module. To demonstrate the effectiveness of the two branches of the CTL module, we conduct ablation experiments on K400 [3]. The base model is a simple model that replaces the CTL into a 3×3 convolution. As shown in Table 1, the TFConv branch (branch1) achieves 62.8% Top1 accuracy. The accuracy of the IOI branch (branch2) is 69.6%, which is 6.8% higher than the TFConv branch. It is because TFConv branch only concentrates on extracting temporal importance of multiple channels using 1×1 convolution while the IOI branch captures both the temporal importance and spatial relation using large kernels. As shown in the last row, when combining the two branches, the performance of the proposed *SqueezeTime* can reach 71.6%, which is 1.0% higher than only using the IOI branch and 4.4% higher than the base model.

Effectiveness analysis of the temporal focus convolution (TFC). The TFConv branch of the CTL module is to model the temporal importance of multiple channels. In this experiment, we analyze by only using the TFConv branch of the CTL Module. As shown in Table 3, when not using the temporal focus convolution (TFC), *i.e.*, only using 1×1 convolution, the accuracy is only 49.9%. When using the temporal focus convolution, the accuracy is increased by 12.9%. It demonstrates that temporal focus convolution is important when squeezing the temporal axis into channels.

Ablation analysis of the IOI branch of CTL Module. We make an ablation analysis of the IOI branch in Table 2. In this experiment, we make an ablation on the model only using the IOI branch (*i.e.*, the third row of Table 1). As shown in Table 2, the base model in the first row is only a 3×3 convolution (see the bottom part of Figure 3 (c)) that simply maps the input channels into output channels. We then gradually add IOI, temporal focus convolution, and temporal position encoding into the base model. First, the performance of the base model is 67.2%. After adding the top branch of IOI, *i.e.*, the second row of Table 2, the Top1 accuracy is increased to 68.1% ($+0.9\%$), which

Table 5: Impact of different channel expansion of *SqueezeTime*.

Channel factor	**0.5**	**0.75**	**1.0**	**1.25**
FLOPs (G)	1.9	3.4	5.5	8.1
Tput (clips / s)	1964	1186	903	617
Top1 Acc (%)	67.8	69.8	71.6	71.9

Table 6: Latency of models on CPU of a modern mobile phone. Top1 is on K400 dataset. 'LAT' is latency.

Model	I3d	MoViNet	TSN	E3D	TSM	TimeSFormer	SqueezeTime
Backbone	Res18	A0	Res34	XS	Res34	ViT-B	-
Top1(%)	65.3	65.8	67.7	68.6	69.4	70.4	**71.6**
LAT(ms)	869	1282	1239	819	688	960	**139**

Table 7: Performance comparison of multiple lightweight methods on K400 [3] dataset. CPU latency (LAT, ms), GPU throughput (Tput, clips / s), Top1 accuracy, and Top5 accuracy are reported. We retrain and test these methods using mobile settings, *i.e.*, low FLOPs, low CPU Latency, and high GPU throughput, by adopting a smaller backbone, lower clip resolution, or fewer frames to build comparable models. The latency and throughput are tested on an Intel(R) Xeon(R) Gold 6278C@2.60GHz CPU (batch size of 1) and an NVIDIA Tesla V100 GPU (using maximum batch size). The above settings are the same for the following several tables.

Model	**Param(M)**	**FLOPs(G)**	**Frames**	**LAT(CPU)↓**	**Tput(GPU)↑**	**Top1(%)**	**Top5(%)**
I3d-R18 [7]	33.4	46.0	32×3×10	450	128	65.3	86.6
MoViNet-A0 [12]	3.1	2.7	50×1×1	505	108	65.8	87.4
TSM-R18 [14]	11.2	19.0	8×8×10	186	429	65.9	86.6
X3d-XS [2]	3.80	0.9	4×3×10	219	441	66.6	86.7
TADA-R18 [19]	14.2	19.2	8×3×10	767	76	66.9	87.5
TSN-R34 [32]	21.5	91.8	25×25×10	636	108	67.7	88.0
R2plus1d-R18 [11]	33.5	30.4	8×3×10	510	133	68.1	87.7
C2d-R34 [29]	21.5	17.8	8×3×10	205	455	68.5	88.0
SlowOnly-R18 [8]	32.0	35.9	8×3×10	303	164	68.6	88.3
E3d-XS [13]	2.2	2.4	16×3×10	313	203	68.6	88.3
Slowfas-R18 [8]	33.6	25.2	32×3×10	604	116	69.0	88.9
TIN-R34 [31]	21.5	29.4	8×8×1	345	274	69.1	88.5
TSM-R34 [14]	21.5	29.4	8×8×10	238	307	69.4	88.8
TANet-R18 [16]	11.8	19.1	8×3×8	192	379	69.6	88.7
R2plus1d-R34 [11]	63.8	53.1	8×3×10	748	82	69.8	88.4
TPN-R18 [28]	38.1	36.7	8×3×10	344	158	70.2	89.3
MViTv2-Small [21]	34.5	4.2	4×3×10	176	517	68.8	88.2
VideoSwin-Tiny [22]	28.2	5.6	4×3×10	162	505	69.9	88.9
TimeSformer-Base [20]	86.0	23.2	4×3×10	196	205	70.4	89.2
SqueezeTime (Ours)	28.7	5.5	16×3×10	**123**	**903**	**71.6**	**89.3**

demonstrates the cross-temporal object relation is important for the model. Then after replacing the first 3×3 convolution of the top part into a 3×3 TFC (*i.e.*, the third row), the Top1 accuracy is further increased to 69.2% ($+0.9\%$). Finally, the temporal position encoding further improves the performance by 0.4%, which indicates that temporal order information is also necessary for the model to capture temporal representations.

Impact of the number of input frames. Here we investigate the effects of changing the number of input frames. When changing the different number of input frames, the input channel number of the 5×5 convolution of the stem layer (see the top part of Figure 3) is changed accordingly. The sampling interval of frames 4, 8, 16 and 32 are 12, 8, 4 and 2, respectively. Other models and training settings are unchanged. As shown in Table 4, using 4 and 8 frames, the Top1 accuracies are 70.7% and 70.9%, respectively. When using 16 frames, the performance is increased to 71.6% ($+0.7\%$). The 32 frames have the best 71.7% Top1 accuracy. Considering the data loading time consuming of 32 frames is much more than 16 frames, we finally use 16 input frames in the proposed *SqueezeTime*.

Impact of the expansion of channels of *SqueezeTime*. Note we squeeze the temporal axis of a video sequence into the spatial channel dimension, thus we research the effect of channel numbers of the proposed *SqueezeTime* in Table 5. The base channel factor 1.0 represents that the output channels of four stages are $[256, 512, 1024, 2048]$. Other factors are changed according to the factor. The number of channels are changed according to the expansion factor. As shown in the table, the expansion factor 1.0 achieves the best balance between the accuracy and GPU throughput.

Speed evaluation of different methods on mobile devices. As shown in Table 6, we provided the latencies of multiple methods on the mobile device. As shown in the table, the proposed SqueezeTime is much more efficient for the constrained hardware, *e.g.*, six times faster (only 139ms) than TimesFormer (960ms), on the mobile device.

4.4 Comparisons with the State of the Arts

In this section, we conduct comprehensive experiments to compare our *SqueezeTime* with state-of-the-art methods in mobile video analysis settings.

Table 8: Performace comparison of lightweight methods on K600 [4] dataset. LAT: CPU Latency (ms), Tput: GPU Throughput (clips/s).

Model	Param(M)	FLOPs(G)	Frames	LAT(CPU)↓	Tput(GPU)↑	Top1(%)	Top5(%)
I3d-R18 [7]	33.4	46.0	32×3×10	450	128	70.7	90.4
TSN-R34 [32]	21.5	91.8	25×25×10	636	108	71.4	90.2
MoViNet-A0 [12]	3.1	2.7	50×1×1	505	108	71.5	90.4
X3d-XS [2]	3.8	0.9	4×3×10	219	441	71.5	90.6
TIN-R34 [31]	21.5	29.4	8×8×1	345	274	71.6	90.7
C2d-R34 [29]	21.5	17.8	8×3×10	205	455	72.1	90.8
SlowOnly-R18 [8]	32.0	35.9	8×3×10	303	164	72.2	91.1
E3d-XS [13]	2.2	2.4	16×3×10	313	203	72.9	91.5
Slowfast-R18 [8]	33.6	25.2	32×3×10	604	116	73.5	91.9
TSM-R34 [14]	21.5	29.4	8×8×10	238	307	73.5	91.3
TANet-R18 [16]	11.8	19.1	8×8×10	192	379	73.8	91.9
TPN-R18 [28]	38.1	36.7	8×3×10	344	158	73.8	91.9
TADA-R18 [19]	14.2	19.2	8×3×10	767	77	73.9	92.0
R2plus1d-R18 [11]	33.5	30.4	8×3×10	510	133	74.0	92.0
R2plus1d-R34 [11]	63.8	53.1	8×3×10	748	82	75.5	92.6
TimeSformer-Base [20]	86.0	23.2	4×3×10	196	205	73.5	91.4
VideoSwin-Tiny [22]	28.2	5.6	4×3×10	162	505	73.6	91.4
MViTv2-Small [21]	34.5	4.2	4×3×10	176	517	73.6	91.7
SqueezeTime (Ours)	28.7	5.5	16×3×10	**123**	**903**	**76.0**	92.5

Comparison of action recognition on K400 [3]. As shown in Table 7, we compare the proposed *SqueezeTime* with 19 state-of-the-art methods. Note we aim to build a lightweight and fast backbone, *i.e.*, with low CPU latency, high GPU throughput, and reliable accuracy, for mobile video analysis, thus we retrain and test these compared models using mobile settings for a fair comparison. We convert these models into mobile versions by adopting smaller backbones, using lower video resolution, or using fewer frames. As shown in Table 7, the top rows are CNN-based models and the middle rows are transformer-based models. The best comprehensive CNN network and transformer network are TANet [16] and VideoSwin [22] respectively. The proposed *SqueezeTime* performs better than all compared models in terms of CPU latency, GPU Throughput, and Top1 accuracy. For example, it exceeds the best compared transformer-based model VideoSwin by 1.7% Top1 accuracy but with $\sim 25\%$ lower CPU latency and $\sim 80\%$ higher GPU throughput and exceeds the best compared CNN-based model TANet by 2.0% with $\sim 25\%$ lower CPU latency and $\sim 140\%$ higher GPU throughput. From the table, we can also find the 2D-based CNN models, *e.g.*, TANet and TSM are more mobile friendly (faster) than the 3D-based CNN models, *e.g.*, I3d, X3d, MoViNet, and E3d. Although the transformer-based models show competitive performance compared with these 2D-based CNN models, the proposed *SqueezeTime* using 2D convolution can also beat them in mobile settings, which demonstrates the effectiveness of our model.

Comparison of action recognition on K600 [4]. As shown in Table 8, We compare the proposed *SqueezeTime* with state-of-the-art models on K600. As shown in the table, our model significantly outperforms other methods with at least 0.5% Top1 accuracy improvement, $\sim 25\%$ CPU speed improvement, and $\sim 80\%$ GPU speed improvement. Note that although R2plus1d with ResNet34 backbone achieves the best Top1 accuracy, *i.e.*, 75.5%, among all compared methods on K600, its CPU speed, and GPU speed are only $\sim 1/6$ and $\sim 1/9$ of our proposed *SqueezeTime*.

Comparison of action recognition on HMDB51 [5]. As shown in Table 9, we compare our model with 16 other methods on HMDB51. From the table, we can find that TPN reaches the best Top1 accuracy (0.2% better than ours), but its CPU speed and GPU speed are only approximate $1/2$ and $1/4$ of our model. Our proposed *SqueezeTime* also shows competitive performance on this dataset.

Comparison of action detection on AVA2.1 [43]. In this experiment, we demonstrate the effectiveness of *SqueezeTime* on downstream AVA action detection task in Table 10. For the AVA2.1 dataset, spatial-temporal labels are provided for one frame per second, and each person was annotated with a bounding box and his actions. We follow the standard protocol of [8] to train the model and evaluate the performance on 60 chasses. We report the mean average precision (mAP) over 60 classes, using a frame-level IoU threshold of 0.5. We experiment using mmaction2 [47] framework and replace the backbone of [8] using the proposed *SqueezeTime*. To align the deep features of the backbone, we simply reshape the output of *SqueezeTime*, *i.e.*, $(C_{out}, \mathbf{F}_h, \mathbf{F}_w)$, to $(C_{out}//16, 16, \mathbf{F}_h, \mathbf{F}_w)$, where C_{out}, $\mathbf{F}_h$ and $\mathbf{F}_w$ are the channel number, height and width of the feature map. The number 16 is the temporal dimension of the reshaped feature. The proposed *SqueezeTime* shows competitive

Table 9: Performace comparison of action recognition on HMDB51 [5] dataset (Pretrained on K400). LAT: CPU Latency (ms), Tput: GPU Throughput (clips/s).

Model	Param(M)	FLOPs(G)	Frames× Views	LAT(CPU)↓	Tput(GPU)↑	Top1(%)	Top5(%)
X3d-XS [2]	3.8	0.9	4×3×2	219	441	57.1	85.4
E3d-XS [13]	2.2	1.2	4×3×2	211	415	57.7	86.3
SlowOnly-R18 [8]	32.0	18.0	8×3×2	268	321	61.2	88.5
I3d-R18 [7]	33.4	46.0	32×3×2	450	128	61.3	87.3
TSN-R34 [32]	21.5	91.8	8×3×2	636	108	61.4	89.1
TSM-R18 [14]	11.2	19.0	8×3×2	186	429	61.6	88.4
R2plus1d-R34 [11]	63.8	53.1	8×3×2	748	82	62.0	86.8
TIN-R34 [31]	21.5	29.4	8×3×2	345	274	63.0	87.7
TANet-R18 [16]	11.8	19.1	8×3×2	192	379	63.8	89.8
C2d-R34 [29]	21.5	17.8	8×3×2	205	455	63.9	90.9
Slow-fast-R18 [8]	33.8	19.3	16×3×2	473	157	64.1	89.0
E3d-XS [13]	2.2	2.4	16×3×2	313	203	65.0	91.1
TPN-R18 [28]	37.4	28.1	8×3×2	256	203	65.2	89.4
VideoSwin-Tiny [22]	28.2	5.6	4×3×2	162	505	61.1	86.4
MViTv2-Small [21]	34.5	4.2	4×3×2	176	517	62.2	88.2
Timesformer-Base [20]	86.0	23.2	4×3×2	196	205	63.5	89.8
SqueezeTime (Ours)	28.7	5.5	16×3×2	**123**	**903**	**65.6**	90.3

Table 10: Action detection results on AVA2.1 [43]. We train and test these methods using the same settings as [8].

Model	Frame	mAP	Time
SlowOnly-R18 [8]	8	10.8	3.9 ms
X3d-XS [2]	16	13.6	9.8 ms
E3d-XS [13]	16	14.8	7.3 ms
VideoSwin [22]	16	12.7	8.6 ms
I3d-R18 [7]	16	15.2	6.3 ms
R2plus1d-R34 [11]	16	15.4	11.2 ms
SqueezeTime (Ours)	16	15.1	**3.4** ms

Table 11: Temporal action localization results of models on THUMOS14 [44] dataset. We measure the performance by mAP at different tIOU thresholds and average mAP in [0.3:0.1:0.7]. We use the framework of AFSD [46] to train and test these methods. 'Time' is the average time of predicting a test sample.

Model	0.3	0.4	0.5	0.6	0.7	Avg	Time
I3d-R18 [7]	42.7	37.6	31.4	23.2	14.8	29.9	10.9s
X3d-XS [2]	45.6	40.4	32.5	22.6	13.4	30.9	2.9s
E3d-XS [13]	46.6	41.6	33.7	24.8	15.3	32.4	1.4s
SqueezeTime (Ours)	48.3	43.0	34.8	24.7	13.8	**32.7**	**1.2**s

performance in Table 10. Although the mAP of our model is 0.3 lower than the best compare R2plus1d-R34 [11], the average test time on the test set of our model is only $\sim 1/3$ of it, which is much more efficient for the mobile usage.

Comparison of action detection on THUMOS14 [44]. In this part, we investigate the effectiveness of the proposed *SqueezeTime* on the action detection dataset of THUMOS14 [44]. In this experiment, we compare the proposed model with 3 popular models based on the framework of [46]. We replace the original backbone of [46] using these models and remove the boundary refinement mechanism, *i.e.*, the models finally contain only a backbone and a prediction head. We adjust the number of input frames, and resolution of videos of these models to make fair comparisons in mobile settings. The other training and test settings are as default as [46]. As for the proposed *SqueezeTime*, we make some designs to make it suitable for action detection tasks with a long temporal sequence. For the 256 input frames, we split them along the temporal dimension using a temporal sliding window of 16 frames and the stride is 8. Then we get 32 temporal clips with the frame number of 16 and we feed them to *SqueezeTime* in parallel to get features of a temporal dimension of 32. Then the features are fed into the detection head to predict the result. As shown in Table 11, the proposed *SqueezeTime* can exceed all 3 compared methods in terms of the average mAP and test time. It is 0.3 better than the recently proposed E3d-XS but using $\sim 20\%$ fewer test time, which demonstrates the effectiveness of *SqueezeTime* as a backbone for the downstream action detection task.

5 Conclusion

In this paper, we concentrate on building a lightweight and fast model for mobile video analysis. Different from current popular video models that regard time as an extra dimension, we propose to squeeze the temporal axis of a video sequence into the spatial channel dimension, which saves a great amount of memory and computation consumption. To remedy the performance drop caused by the squeeze operation, we elaborately design an efficient backbone *SqueezeTime* with a stack of efficient Channel-Time Learning Block (CTL), which consists of two complementary branches to restore and excavate temporal dynamics. Besides, we make comprehensive experiments to compare a quantity of state-of-the-art methods in mobile settings, which shows the superiority of the proposed *SqueezeTime*, and we hope it can foster further research on mobile video analysis.

References

[1] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *CVPR*, pages 770–778, 2016.

[2] Christoph Feichtenhofer. X3d: Expanding architectures for efficient video recognition. In *CVPR*, pages 203–213, 2020.

[3] Will Kay, Joao Carreira, Karen Simonyan, Brian Zhang, Chloe Hillier, Sudheendra Vijayanarasimhan, Fabio Viola, Tim Green, Trevor Back, Paul Natsev, et al. The kinetics human action video dataset. *arXiv preprint arXiv:1705.06950*, 2017.

[4] Joao Carreira, Eric Noland, Andras Banki-Horvath, Chloe Hillier, and Andrew Zisserman. A short note about kinetics-600. *arXiv preprint arXiv:1808.01340*, 2018.

[5] Hildegard Kuehne, Hueihan Jhuang, Estibaliz Garrote, Tomaso Poggio, and Thomas Serre. Hmdb: a large video database for human motion recognition. In *ICCV*, pages 2556–2563, 2011.

[6] Du Tran, Lubomir Bourdev, Rob Fergus, Lorenzo Torresani, and Manohar Paluri. Learning spatiotemporal features with 3d convolutional networks. In *ICCV*, pages 4489–4497, 2015.

[7] Joao Carreira and Andrew Zisserman. Quo vadis, action recognition? a new model and the kinetics dataset. In *CVPR*, pages 6299–6308, 2017.

[8] Christoph Feichtenhofer, Haoqi Fan, Jitendra Malik, and Kaiming He. Slowfast networks for video recognition. In *ICCV*, pages 6202–6211, 2019.

[9] Kensho Hara, Hirokatsu Kataoka, and Yutaka Satoh. Can spatiotemporal 3d cnns retrace the history of 2d cnns and imagenet? In *CVPR*, pages 6546–6555, 2018.

[10] Saining Xie, Chen Sun, Jonathan Huang, Zhuowen Tu, and Kevin Murphy. Rethinking spatiotemporal feature learning: Speed-accuracy trade-offs in video classification. In *ECCV*, pages 305–321, 2018.

[11] Du Tran, Heng Wang, Lorenzo Torresani, Jamie Ray, Yann LeCun, and Manohar Paluri. A closer look at spatiotemporal convolutions for action recognition. In *CVPR*, pages 6450–6459, 2018.

[12] Dan Kondratyuk, Liangzhe Yuan, Yandong Li, Li Zhang, Mingxing Tan, Matthew Brown, and Boqing Gong. Movinets: Mobile video networks for efficient video recognition. In *CVPR*, pages 16020–16030, 2021.

[13] Junyan Wang, Zhenhong Sun, Yichen Qian, Dong Gong, Xiuyu Sun, Ming Lin, Maurice Pagnucco, and Yang Song. Maximizing spatio-temporal entropy of deep 3d cnns for efficient video recognition. *ICLR*, 2023.

[14] Ji Lin, Chuang Gan, and Song Han. Tsm: Temporal shift module for efficient video understanding. In *ICCV*, pages 7083–7093, 2019.

[15] Limin Wang, Zhan Tong, Bin Ji, and Gangshan Wu. Tdn: Temporal difference networks for efficient action recognition. In *CVPR*, pages 1895–1904, 2021.

[16] Zhaoyang Liu, Limin Wang, Wayne Wu, Chen Qian, and Tong Lu. Tam: Temporal adaptive module for video recognition. In *ICCV*, pages 13708–13718, 2021.

[17] Yulin Wang, Zhaoxi Chen, Haojun Jiang, Shiji Song, Yizeng Han, and Gao Huang. Adaptive focus for efficient video recognition. In *ICCV*, pages 16249–16258, 2021.

[18] Wangmeng Xiang, Chao Li, Biao Wang, Xihan Wei, Xian-Sheng Hua, and Lei Zhang. Spatiotemporal self-attention modeling with temporal patch shift for action recognition. In *ECCV*, pages 627–644, 2022.

[19] Ziyuan Huang, Shiwei Zhang, Liang Pan, Zhiwu Qing, Mingqian Tang, Ziwei Liu, and Marcelo H Ang Jr. Tada! temporally-adaptive convolutions for video understanding. In *ICLR*, 2022.

[20] Gedas Bertasius, Heng Wang, and Lorenzo Torresani. Is space-time attention all you need for video understanding? page 4, 2021.

[21] Yanghao Li, Chao-Yuan Wu, Haoqi Fan, Karttikeya Mangalam, Bo Xiong, Jitendra Malik, and Christoph Feichtenhofer. Mvitv2: Improved multiscale vision transformers for classification and detection. In *CVPR*, pages 4804–4814, 2022.

[22] Ze Liu, Jia Ning, Yue Cao, Yixuan Wei, Zheng Zhang, Stephen Lin, and Han Hu. Video swin transformer. In *CVPR*, pages 3202–3211, 2022.

[23] Taojiannan Yang, Yi Zhu, Yusheng Xie, Aston Zhang, Chen Chen, and Mu Li. Aim: Adapting image models for efficient video understanding. In *ICLR*, 2023.

[24] Javier Selva, Anders S Johansen, Sergio Escalera, Kamal Nasrollahi, Thomas B Moeslund, and Albert Clapés. Video transformers: A survey. *IEEE TPAMI*, 2023.

[25] Madeline Chantry Schiappa, Naman Biyani, Prudvi Kamtam, Shruti Vyas, Hamid Palangi, Vibhav Vineet, and Yogesh S Rawat. A large-scale robustness analysis of video action recognition models. In *CVPR*, pages 14698–14708, 2023.

[26] Yu Kong and Yun Fu. Human action recognition and prediction: A survey. *IJCV*, 130(5):1366–1401, 2022.

[27] Du Tran, Heng Wang, Lorenzo Torresani, and Matt Feiszli. Video classification with channel-separated convolutional networks. In *ICCV*, pages 5552–5561, 2019.

[28] Ceyuan Yang, Yinghao Xu, Jianping Shi, Bo Dai, and Bolei Zhou. Temporal pyramid network for action recognition. In *CVPR*, 2020.

[29] Xiaolong Wang, Ross Girshick, Abhinav Gupta, and Kaiming He. Non-local neural networks. In *CVPR*, pages 7794–7803, 2018.

[30] Quanfu Fan, Chun-Fu Richard Chen, Hilde Kuehne, Marco Pistoia, and David Cox. More is less: Learning efficient video representations by big-little network and depthwise temporal aggregation. *NeurIPS*, 32, 2019.

[31] Hao Shao, Shengju Qian, and Yu Liu. Temporal interlacing network. In *AAAI*, pages 11966–11973, 2020.

[32] Limin Wang, Yuanjun Xiong, Zhe Wang, Yu Qiao, Dahua Lin, Xiaoou Tang, and Luc Van Gool. Temporal segment networks: Towards good practices for deep action recognition. In *ECCV*, pages 20–36, 2016.

[33] Yan Li, Bin Ji, Xintian Shi, Jianguo Zhang, Bin Kang, and Limin Wang. Tea: Temporal excitation and aggregation for action recognition. In *CVPR*, pages 909–918, 2020.

[34] Zhaoyang Liu, Donghao Luo, Yabiao Wang, Limin Wang, Ying Tai, Chengjie Wang, Jilin Li, Feiyue Huang, and Tong Lu. Teinet: Towards an efficient architecture for video recognition. In *AAAI*, volume 34, pages 11669–11676, 2020.

[35] Zhengwei Wang, Qi She, and Aljosa Smolic. Action-net: Multipath excitation for action recognition. In *CVPR*, pages 13214–13223, 2021.

[36] Anurag Arnab, Mostafa Dehghani, Georg Heigold, Chen Sun, Mario Lučić, and Cordelia Schmid. Vivit: A video vision transformer. In *ICCV*, pages 6836–6846, 2021.

[37] Yizhou Zhao, Zhenyang Li, Xun Guo, and Yan Lu. Alignment-guided temporal attention for video action recognition. *NeurIPS*, 35:13627–13639, 2022.

[38] Shen Yan, Xuehan Xiong, Anurag Arnab, Zhichao Lu, Mi Zhang, Chen Sun, and Cordelia Schmid. Multiview transformers for video recognition. In *CVPR*, pages 3333–3343, 2022.

[39] Jiewen Yang, Xingbo Dong, Liujun Liu, Chao Zhang, Jiajun Shen, and Dahai Yu. Recurring the transformer for video action recognition. In *CVPR*, pages 14063–14073, 2022.

[40] Rameswar Panda Quanfu Fan, Richard Chen. Can an image classifier suffice for action recognition? In *ICLR*, 2022.

[41] Wenhao Wu, Xiaohan Wang, Haipeng Luo, Jingdong Wang, Yi Yang, and Wanli Ouyang. Bidirectional cross-modal knowledge exploration for video recognition with pre-trained vision-language models. In *CVPR*, pages 6620–6630, 2023.

[42] Raghav Goyal, Samira Ebrahimi Kahou, Vincent Michalski, Joanna Materzynska, Susanne Westphal, Heuna Kim, Valentin Haenel, Ingo Fruend, Peter Yianilos, Moritz Mueller-Freitag, et al. The something something video database for learning and evaluating visual common sense. In *ICCV*, pages 5842–5850, 2017.

[43] Chunhui Gu, Chen Sun, David A Ross, Carl Vondrick, Caroline Pantofaru, Yeqing Li, Sudheendra Vijayanarasimhan, George Toderici, Susanna Ricco, Rahul Sukthankar, et al. Ava: A video dataset of spatio-temporally localized atomic visual actions. In *CVPR*, pages 6047–6056, 2018.

[44] YG Jiang, Jingen Liu, A Roshan Zamir, G Toderici, I Laptev, Mubarak Shah, and Rahul Sukthankar. Thumos challenge: Action recognition with a large number of classe. 2014.

[45] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *CVPR*, pages 248–255, 2009.

[46] Chuming Lin, Chengming Xu, Donghao Luo, Yabiao Wang, Ying Tai, Chengjie Wang, Jilin Li, Feiyue Huang, and Yanwei Fu. Learning salient boundary feature for anchor-free temporal action localization. In *CVPR*, pages 3320–3329, 2021.

[47] MMAction2 Contributors. Openmmlab's next generation video understanding toolbox and benchmark. `https://github.com/open-mmlab/mmaction2`, 2020.